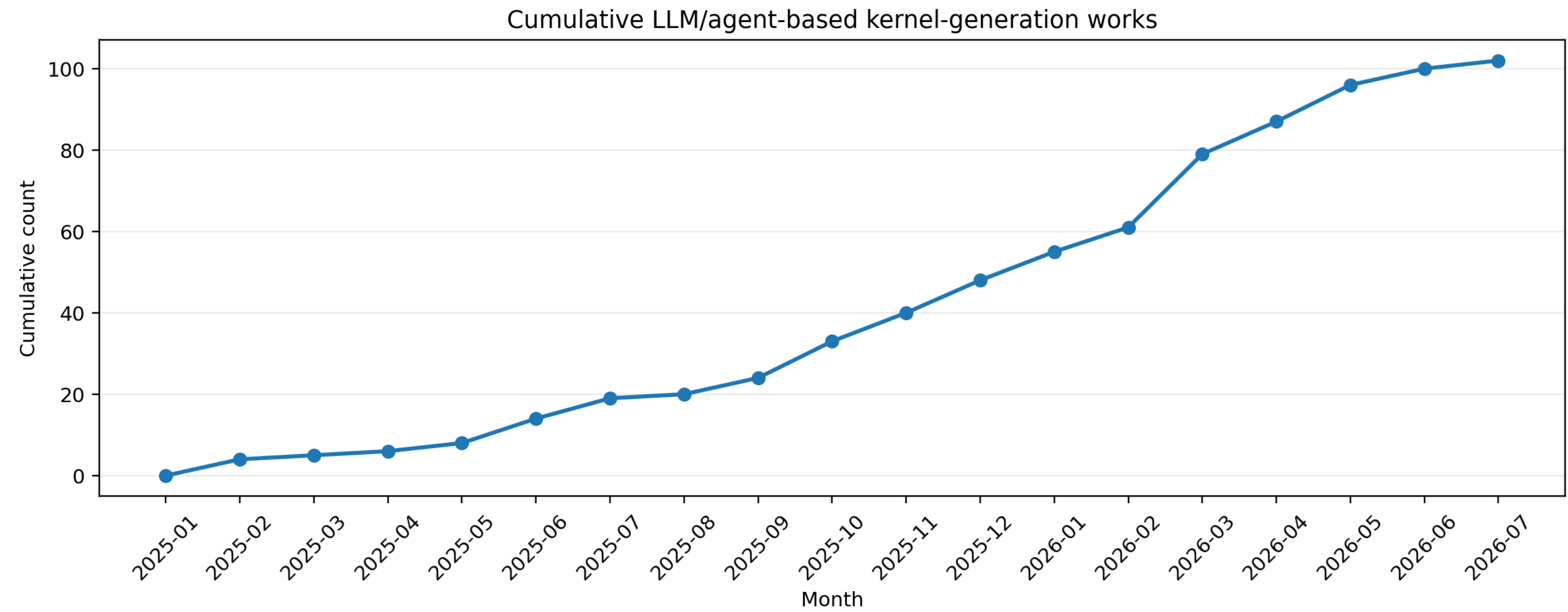


LLM for Kernel generation

Outline

- Background
- Paper and Project
 - QiMeng-Xpiler (OSDI'25)
 - PerfDojo (SC'25)
- Summary

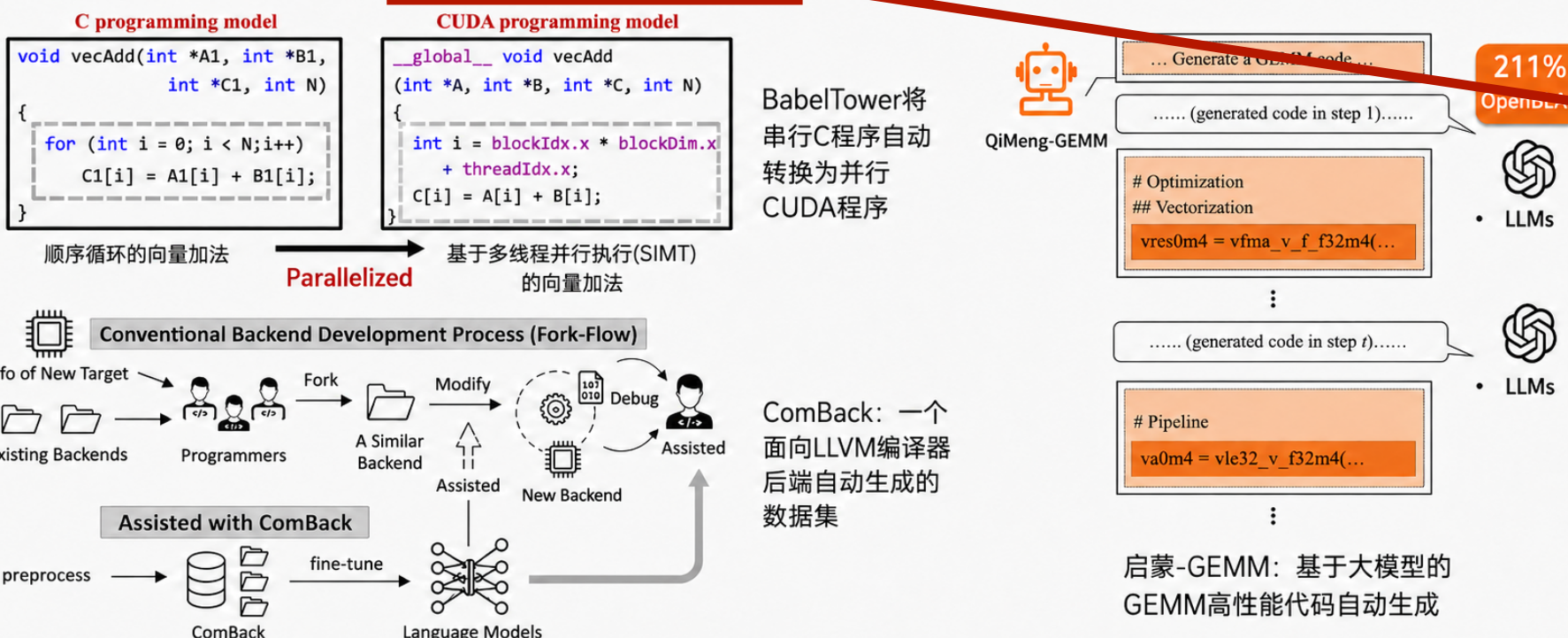


Background

LLM代码生成的应用

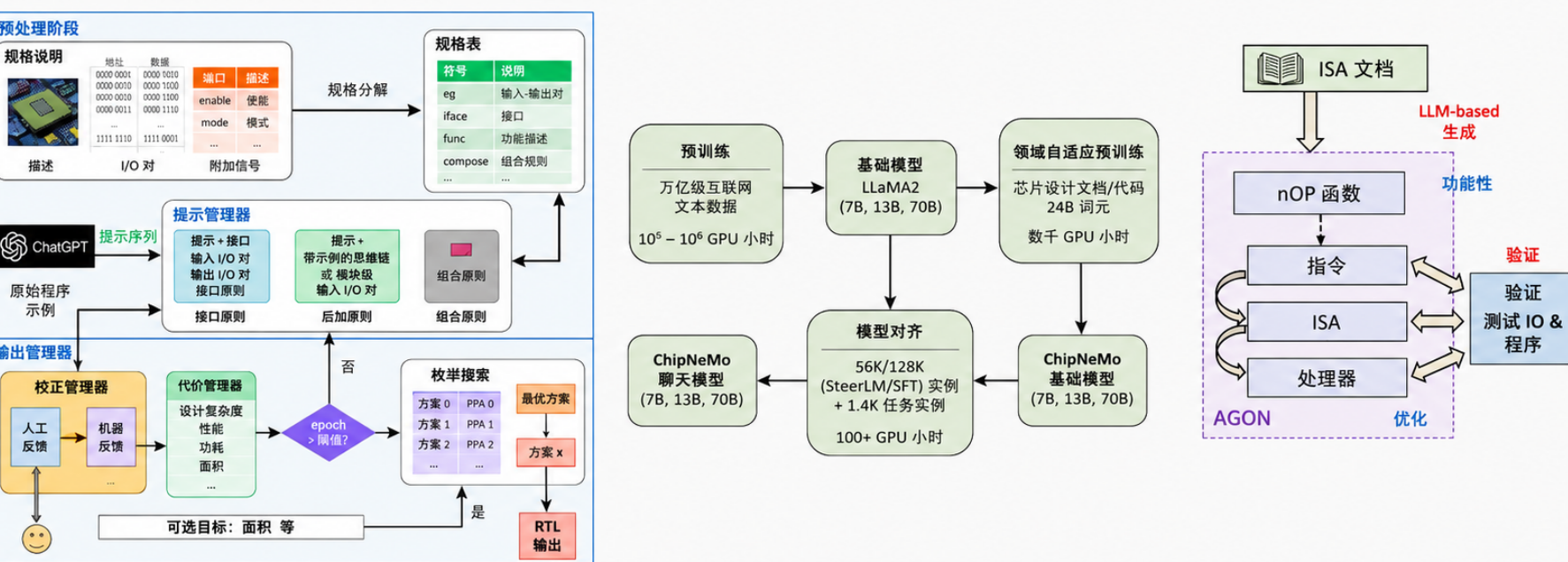
代码生成的应用：系统软件

- 串程序并行化、高性能算子自动生成、编译器后端自动生成



代码生成的应用：芯片设计

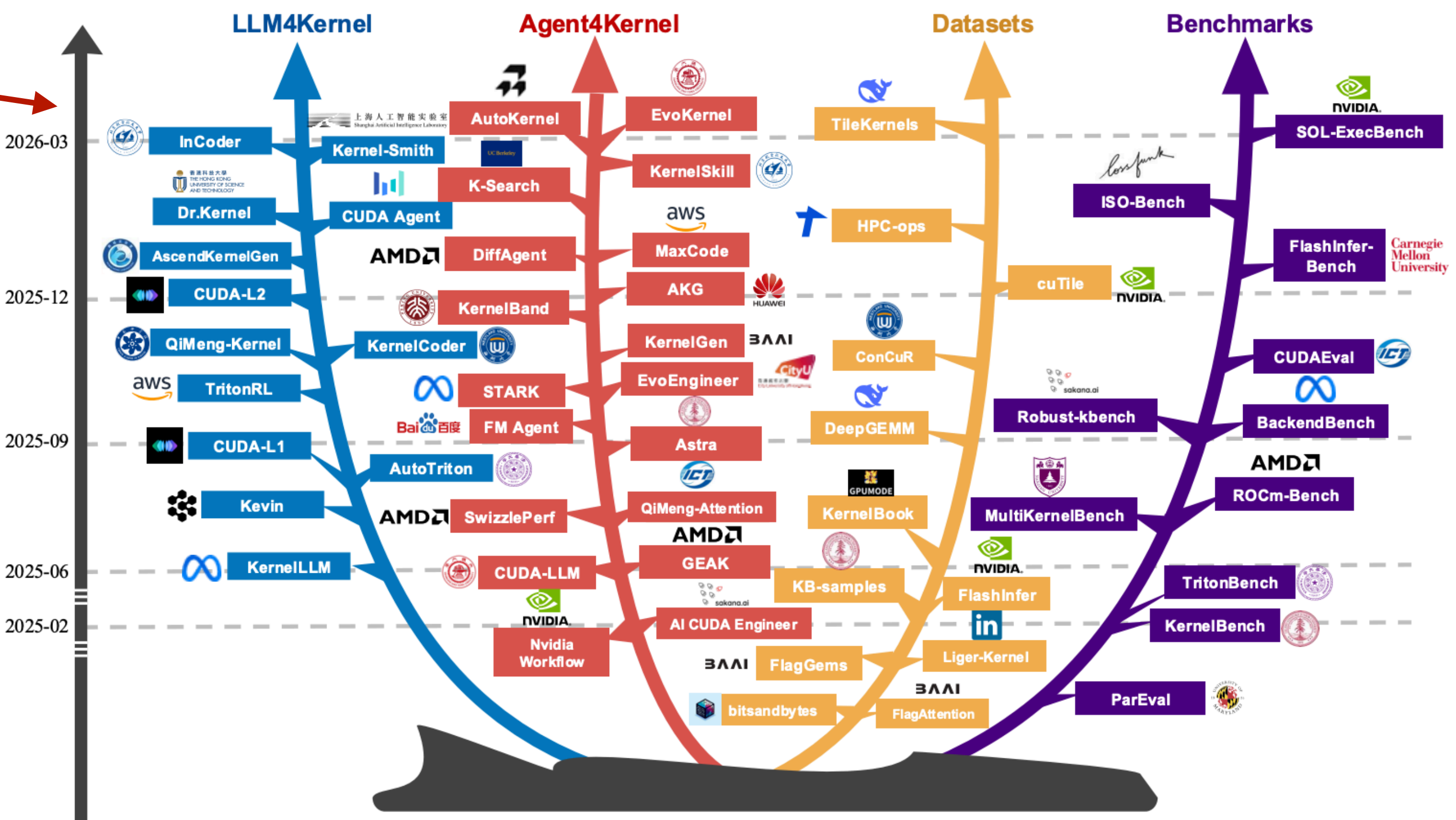
- 芯片设计、知识问答、RTL生成、性能评估



ChipGPT: 使用自然语言交互式设计芯片

ChipNeMo: 基于LLaMA2训练的芯片设计领域专用大模型

AGON: 从指令集文档自动生成CPU原型电路, 实现早期功能和性能评估



- 框架、数据集、评测

结合LLM与符号化方法的张量程序跨硬件转译

QiMeng-Xpiler: Transcompiling Tensor Programs for Deep Learning Systems with a Neural-Symbolic Approach

Shouyang Dong^{1,2,3}, Yuanbo Wen³, Jun Bi³, Di Huang³, Jiaming Guo³, Jianxing Xu^{1,2,3}, Ruibai Xu^{1,2,3},
Xinkai Song³, Yifan Hao³, Ling Li^{4,5}, Xuehai Zhou¹, Tianshi Chen², Qi Guo³, Yunji Chen^{3,5*}

¹*University of Science and Technology of China*, ²*Cambricon Technologies*

³*SKL of Processors, Institute of Computing Technology, Chinese Academy of Sciences*

⁴*Institute of Software, Chinese Academy of Sciences*, ⁵*University of Chinese Academy of Sciences*

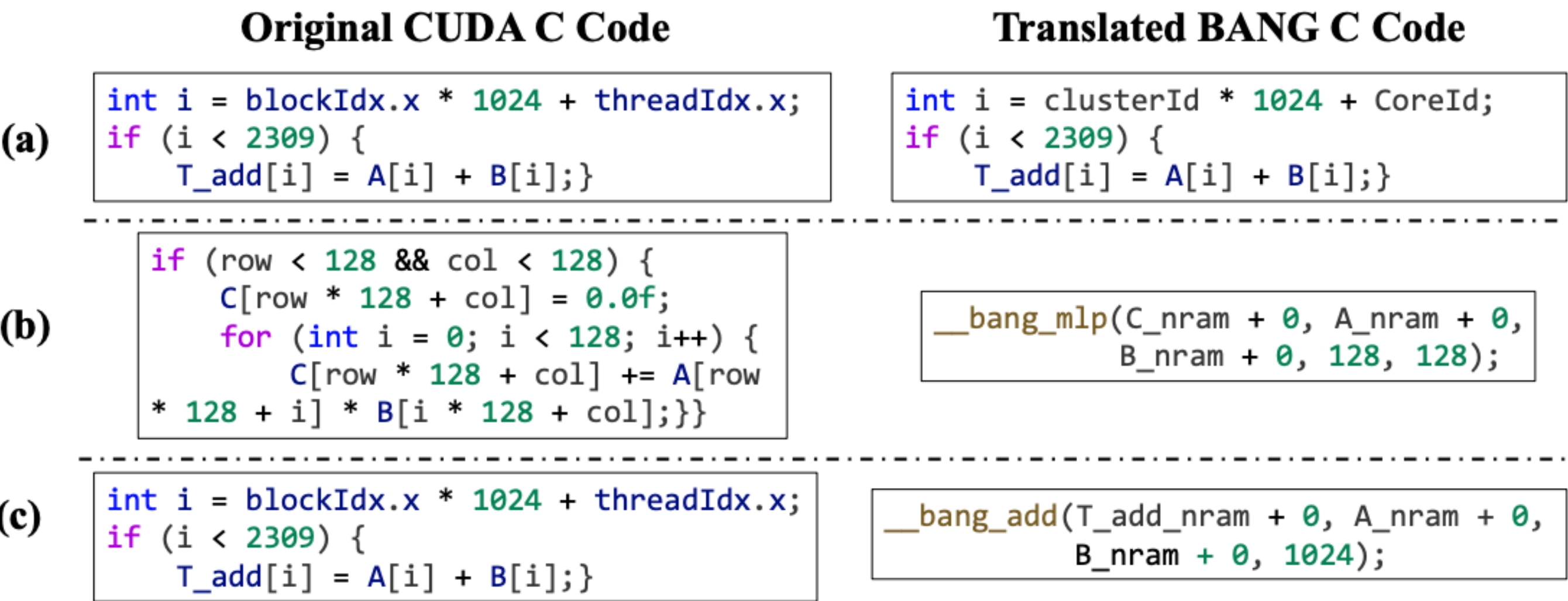
QiMeng-Xpiler: Motivation

Table 1: Comparison of deep learning systems (Intel DL Boost, NVIDIA GPU, AMD MI, and Cambricon MLU) and their programming interfaces. The characteristics fall into three categories: Parallelism, Memory Hierarchy, and Specialized Intrinsics.

Platforms	Interfaces	Categories	Examples
Intel DL Boost	C with VNNI extensions	Specialized Intrinsic	<code>_mm_dpbusds_epi32(...), _mm512_dpbusd_epi32(...)</code>
NVIDIA GPU with Tensor Core	CUDA C	Parallelism	<code>blockIdx, threadIdx</code>
		Memory Hierarchy	<code>registers, __shared__, __global__</code> <code>matrix_a, matrix_b, and accumulator in Tensor Core fragments</code>
		Specialized Intrinsics	<code>wmma::mma_sync(d, a, b, c)</code>
AMD MI with Matrix Core	HIP	Parallelism	<code>blockIdx, threadIdx</code>
		Memory Hierarchy	<code>registers, __shared__, __global__</code> <code>matrix_a, matrix_b, and accumulator in Matrix Core fragments</code>
		Specialized Intrinsics	<code>d = __builtin_amdgcn_mfma_f32_16x16x4f32(a, b, c, ...)</code>
Cambricon MLU	BANG C	Parallelism	<code>taskId</code> for task-level parallelism <code>clusterId, coreId</code> for multi-core parallelism
		Memory Hierarchy	<code>registers, __mlu_shared__, __mlu_device__</code> <code>__nram__, __wram__</code>
		Specialized Intrinsics	<code>__bang_mlp(...), __bang_conv(...)</code>

Why One-shot LLM Transcompilation Fails

Errors align with specific features



Original CUDA C Code

(b)

```
if (row < 128 && col < 128) {
    C[row * 128 + col] = 0.0f;
    for (int i = 0; i < 128; i++) {
        C[row * 128 + col] += A[row
            * 128 + i] * B[i * 128 + col];}}
```

Translated BANG C Code

(b)

```
__bang_mlp(C_nram + 0, A_nram + 0,
            B_nram + 0, 128, 128);
```

Original CUDA C Code

(c)

```
int i = blockIdx.x * 1024 + threadIdx.x;
if (i < 2309) {
    T_add[i] = A[i] + B[i];}
```

Translated BANG C Code

(c)

```
__bang_add(T_add_nram + 0, A_nram + 0,
            B_nram + 0, 1024);
```

Figure 2: The unsuccessful transcompilation examples of GPT-4: (a) Parallelism-related. (b) Memory-related. (c) Instruction-related.

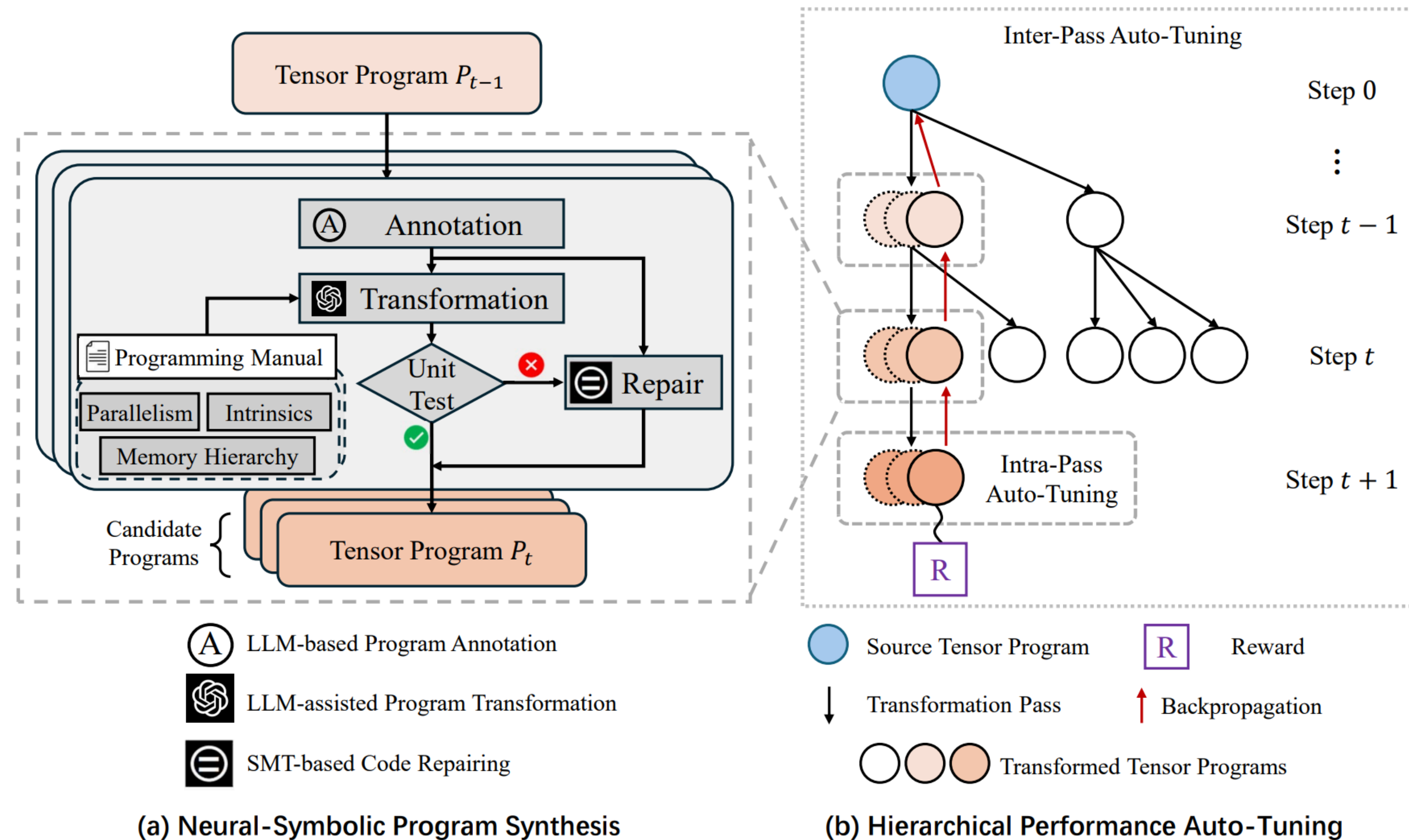
Table 2: Breakdown of the unsuccessful transcompilations produced by GPT-4 based on outcome.(%)

Errors	Compilation			Total	Computation			Total
	Parallelism	Memory	Instruction		Parallelism	Memory	Instruction	
Zero-Shot	3	100	100	100	–	–	–	–
Few-Shot	2.3	27.1	76.5	49.4	97.2	2.8	94.4	92.3

- LLM 单步翻译不稳：可以写出“看起来对”的代码，但常破坏硬件语义。
- 三类错误：并行变量映射、memory space、intrinsic/参数。

QiMeng-Xpiler: Main Idea

LLM proposes code sketch; symbolic tools repair local details



- **核心思想:** 将程序翻译分为多个pass, 每个pass内都由LLM生成 program sketch, 符号化工具修局部错误。
- **性能优化:** pass 内调参, pass 间搜索order

Transformation Passes

Decompose a hard translation into reusable steps

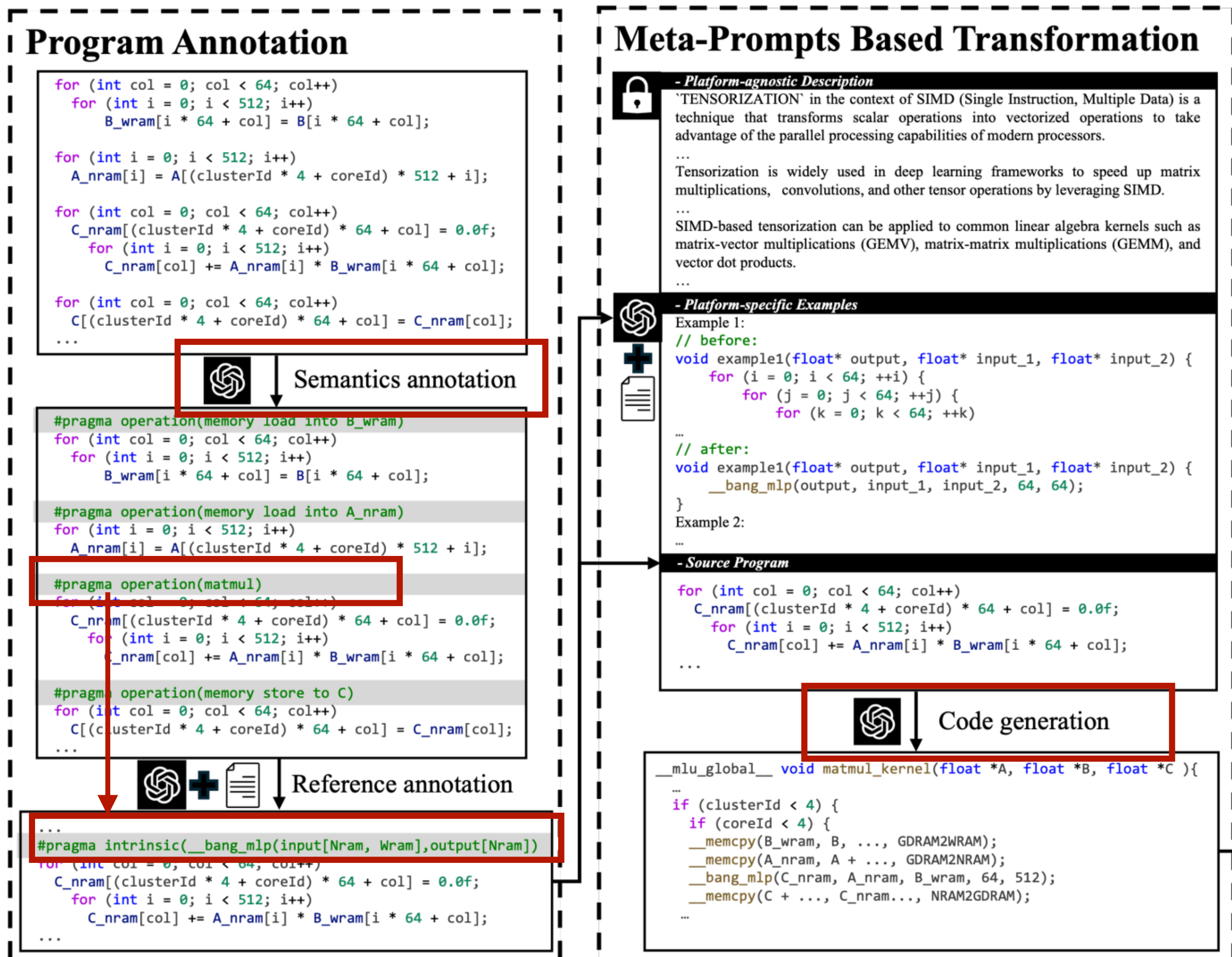
Table 4: The transformation passes and their description.

	Name	Decription
(1)	Loop Recovery	Convert parallel variables to sequential for loops
	Loop Bind	Assign a sequential loop to parallel variables
	Loop Split	Divide a loop into several sub-loops
	Loop Fuse	Merge several loops into a hyper-loop
	Loop Reorder	Change the execution orders of loops
	Loop Expansion	Split a loop body into several loop bodies
	Loop Contraction	Merge the producer in the loop body of consumer
(2)	Cache	Adapt to the memory hierarchy for efficient load/store inputs/outputs
	Pipeline	Pipeline of data load/store and computation
(3)	Tensorize	Replace a specific loop body to leverage special intrinsics
	Detensorize	Restore a specific loop body from special intrinsics

- 把跨平台翻译拆解成：
- Parallelism:将源平台的线程块、线程和循环划分，重新映射到目标平台的 cluster、core 等执行单元
- Memory:把全局内存、片上缓存和局部缓冲区映射到目标平台的存储层次，并插入正确的数据拷贝与同步
- Tensorization:识别矩阵乘、归约等计算模式，将普通循环替换为目标平台的向量或张量指令，并匹配其数据类型、布局和 tile 大小

Meta-prompt Based Transformation

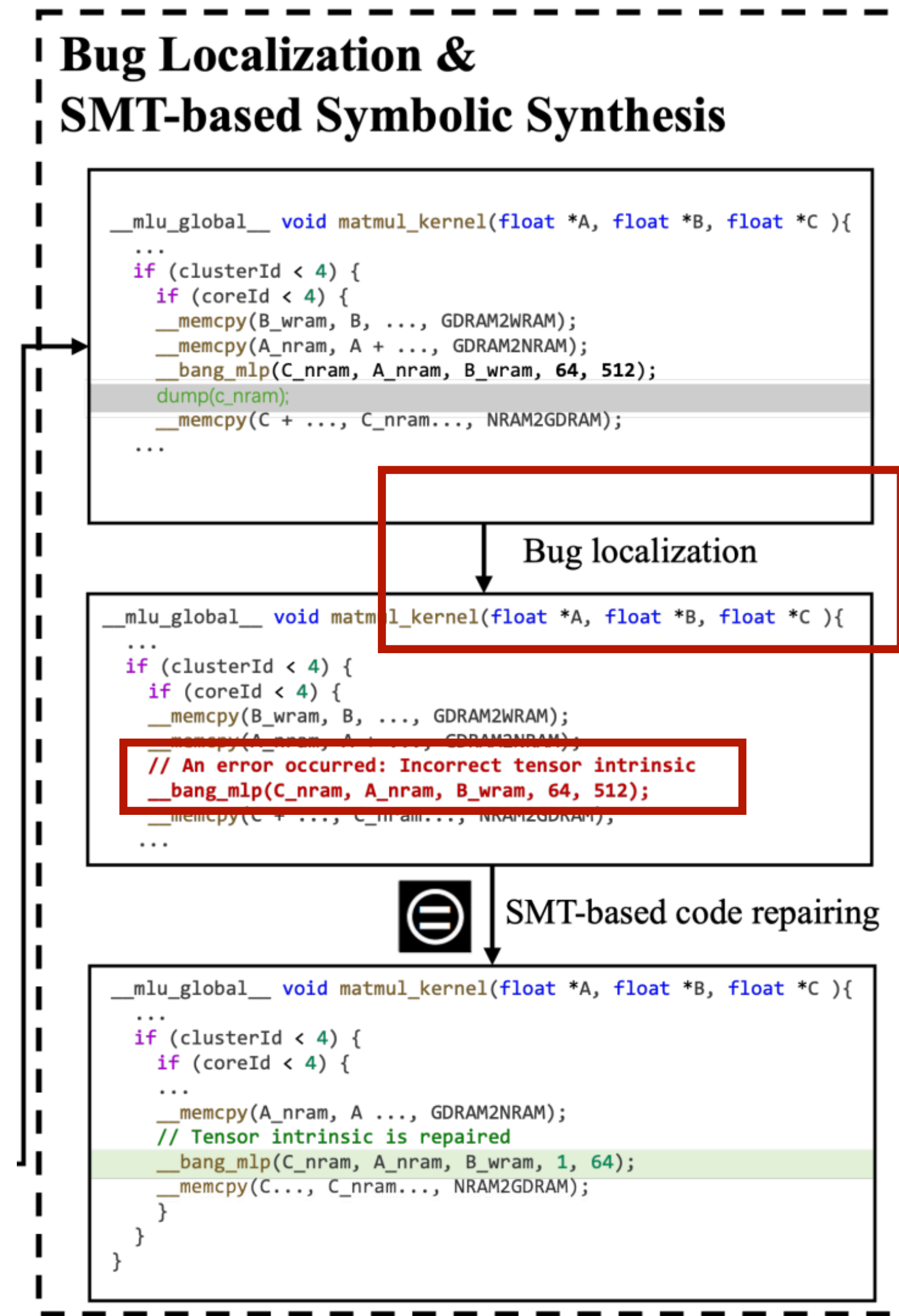
A prompt template is instantiated per pass and per target



- 先识别代码语义：matmul / reduction / elementwise
- meta-prompt = pass说明 + 目标平台示例
- LLM 生成程序框架；具体索引和参数再由测试与 SMT 修正

Bug Localization and Repair

Find the first divergent buffer, then repair only the local code block



(1) Loop Split(index: i, factor: 4)

Source:

```
for (int i = 0; i < 16; i++) {
    B[i] = A[i];}
```

Translated Code Sketch:

```
for (int si = 0; si < ??; si++) {
    for (int ri = 0; ri < 4; ri++) {
        B[si * ?? + ri] = A[si * ?? + ri];}}
```

Constraints:

```
ri - ri' < 4 ∧ ri - ri' > -4
∧ si < bi ∧ si - si' = 0
∧ i = 4 * si + ri ∧ i' = 4 * si' + ri'
∧ B[i] = A[i] ∧ B[i'] = A[i']
```

(2) Cache Read(buffer: A, scope: NRAM)

Source:

```
for (int i = 0; i < 16; i++) {
    B[i] = A[i];}
```

Translated Code Sketch:

```
float A_nram[??];
for (int i = 0; i < 16; i++) { A_nram[i] = A[i];}
for (int in = 0; in < ??; i++) { B[in] = A_nram[in];}
```

Constraints:

```
0 ≤ in ≤ inb
∧ 0 ≤ i < 16
∧ in < i
∧ A_nram[in] = A[i]
∧ B[i] = A_nram[in]
```

Figure 5: The SMT constraints for loop split and cache read.

- 同一输入分别运行源程序和转译程序
- 按执行顺序比较中间 buffer, 二分找到第一次不一致的位置
- 只修写这个 buffer对应的代码块: 索引用 SMT来修复, intrinsic 用 Tenspiler来替换

Hierarchical Auto-tuning

Search both parameters inside a pass and pass sequences across passes

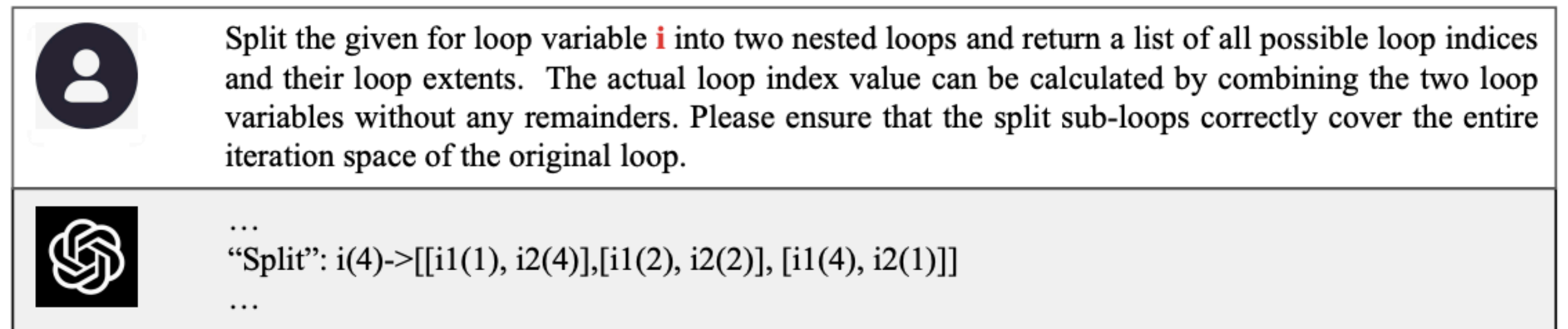
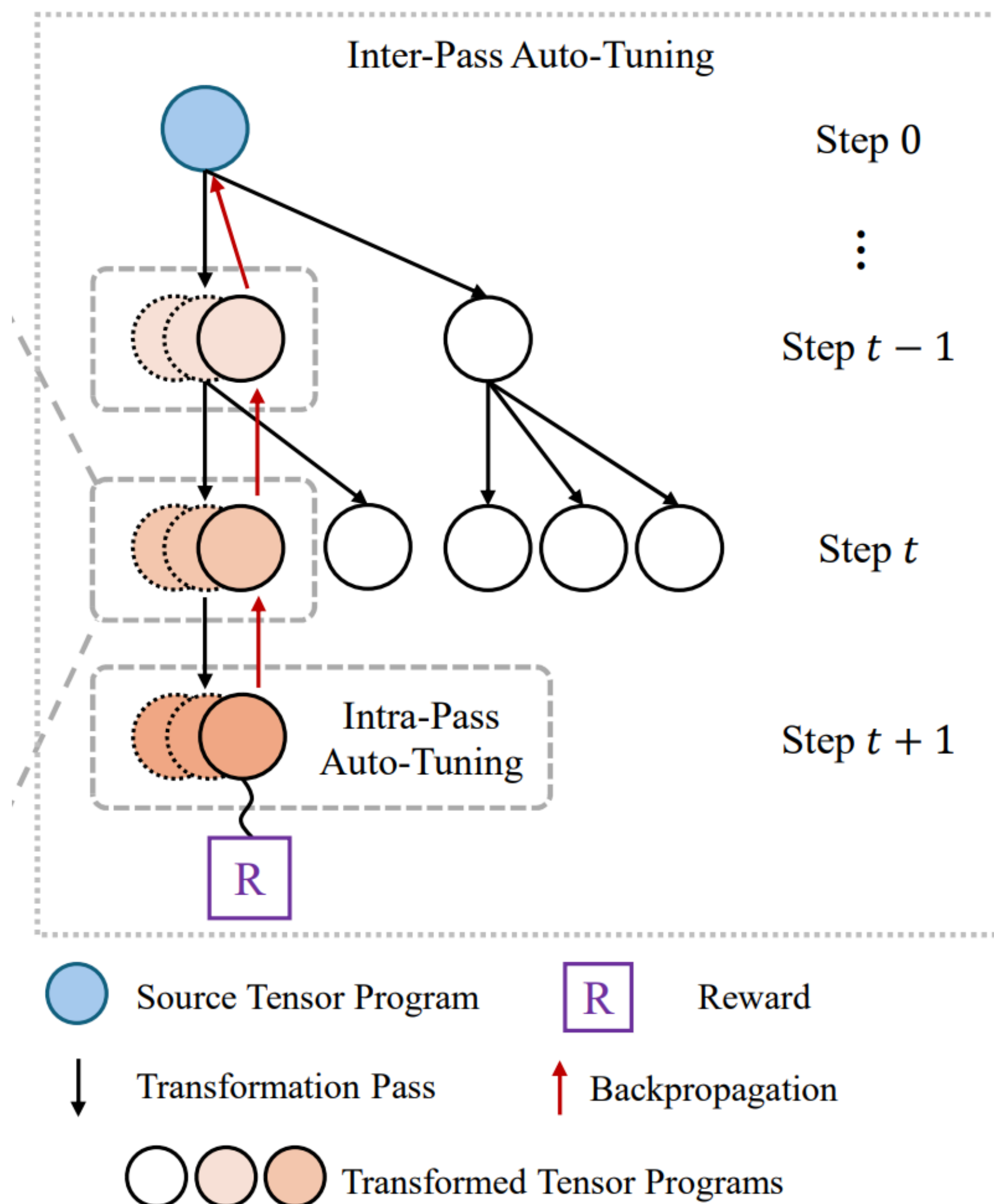


Figure 6: The auto-tuning prompt for loop split.

- **Pass 内：决定“这个优化具体怎么做”**
给定一个 transformation pass，生成不同参数和实现候选，例如 tile 大小、循环划分、线程映射等；编译并实测，保留该 pass 下性能最好的实现。
- **Pass 间：决定“phase order”**
将不同 transformation pass 作为搜索动作，通过 MCTS 探索 pass 的组合与顺序，并利用实际运行时间指导后续搜索。

QiMeng-Xpiler Results

SMT improves correctness; auto-tuning improves practicality

Table 8: Experimental results on different transcompilation directions. (%)

Source	Method	Compilation Accuracy				Computation Accuracy			
		CUDA C	BANG C	Hip	C with VNNI	CUDA C	BANG C	Hip	C with VNNI
CUDA C	GPT-4 Zero-Shot	–	0	82.7	9.5	–	0	82.7	4.2
	OpenAI o1 Zero-Shot	–	0	85.7	61.9	–	0	82.7	60.7
	GPT-4 Few-Shot	–	50.6	97.0	84.5	–	7.7	96.4	30.4
	OpenAI o1 Few-Shot	–	51.8	98.2	85.1	–	48.2	98.2	55.4
	QiMeng-Xpiler w/o SMT	–	82.7	98.2	88.1	–	54.2	98.2	58.3
	QiMeng-Xpiler w/o SMT + Self-Debugging	–	87.5	98.8	89.3	–	54.8	98.2	58.9
	QiMeng-Xpiler	–	100	100	100	–	91.7	100	95.2
BANG C	GPT-4 Zero-Shot	24.4	–	26.8	0	0	–	0	0
	OpenAI o1 Zero-Shot	27.4	–	97.0	9.5	0	–	0	4.2
	GPT-4 Few-Shot	69.0	–	66.1	23.8	6.5	–	6.5	13.1
	OpenAI o1 Few-Shot	71.4	–	97.0	41.7	10.1	–	7.7	23.2
	QiMeng-Xpiler w/o SMT	85.1	–	84.5	47.6	77.4	–	78.6	41.1
	QiMeng-Xpiler w/o SMT + Self-Debugging	88.1	–	88.7	50.6	77.4	–	78.6	41.1
	QiMeng-Xpiler	100	–	100	100	95.8	–	97.0	95.2
Hip	GPT-4 Zero-Shot	97.0	0	–	23.8	97.0	0	–	5.4
	OpenAI o1 Zero-Shot	98.2	0	–	45.8	98.2	0	–	4.2
	GPT-4 Few-Shot	97.0	35.1	–	85.1	97.0	5.4	–	24.4
	OpenAI o1 Few-Shot	98.8	42.3	–	88.7	98.2	9.0	–	30.4
	QiMeng-Xpiler w/o SMT	98.2	60.7	–	65.5	97.6	52.4	–	57.1
	QiMeng-Xpiler w/o SMT + Self-Debugging	98.8	62.5	–	66.1	98.2	52.4	–	57.1
	QiMeng-Xpiler	100	100	–	100	100	86.9	–	96.4
C with VNNI	GPT-4 Zero-Shot	57.1	0	60.1	–	8.3	0	8.9	–
	OpenAI o1 Zero-Shot	66.1	0	97.0	–	10.1	0	96.4	–
	GPT-4 Few-Shot	81.5	41.7	74.4	–	14.3	6.0	12.5	–
	OpenAI o1 Few-Shot	87.5	55.4	97.0	–	51.2	10.7	96.4	–
	QiMeng-Xpiler w/o SMT	95.8	78.0	87.5	–	83.9	58.3	85.7	–
	QiMeng-Xpiler w/o SMT + Self-Debugging	97.0	84.5	89.3	–	83.9	58.3	85.7	–
	QiMeng-Xpiler	100	99.4	100	–	98.2	88.7	99.4	–

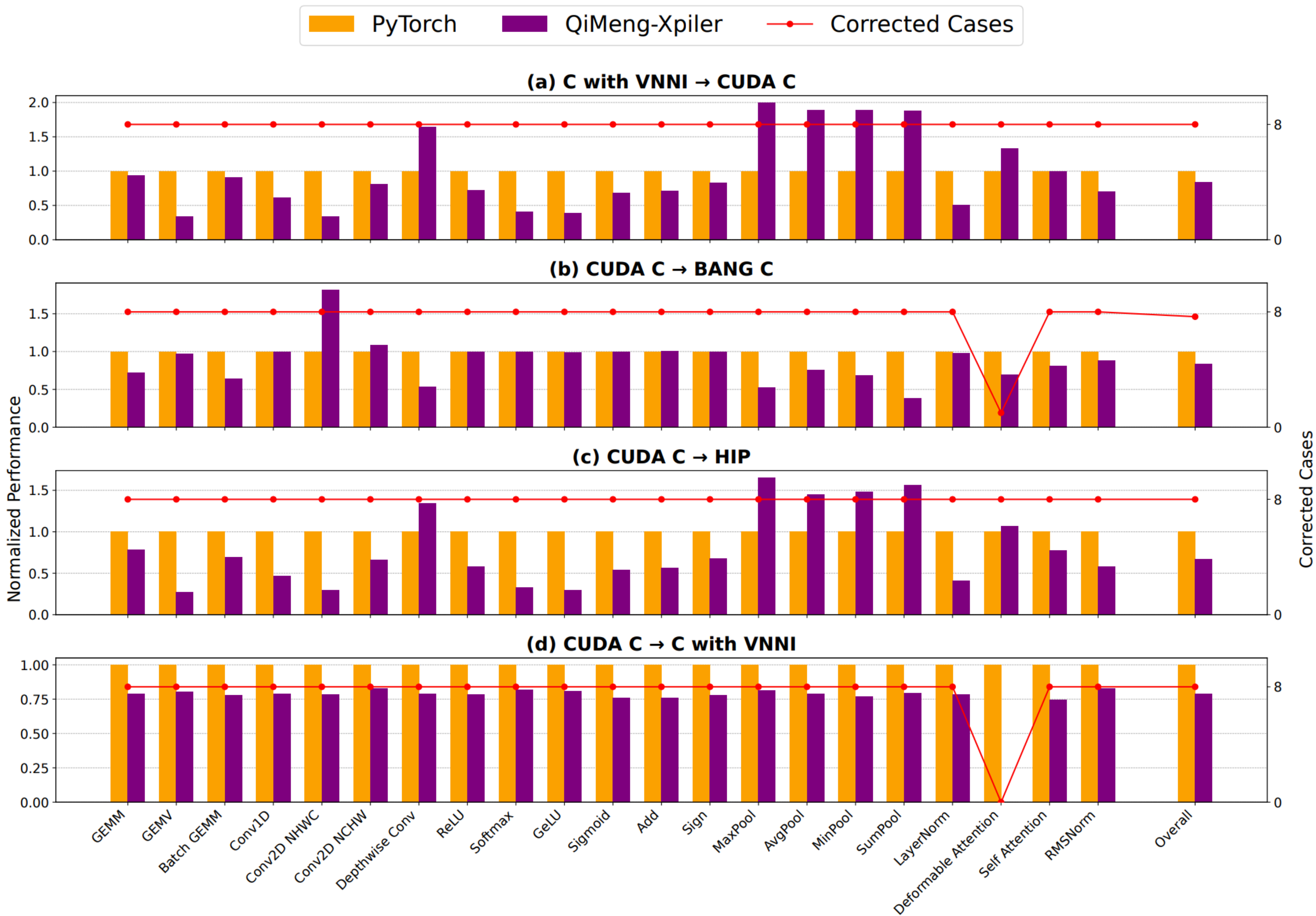
Table 9: Accuracy comparison to rule-based methods.(%)

Direction	Method	Compilation	Computation
CUDA C → HIP	Hipify	85.7	85.7
	QiMeng-Xpiler	100	100
C → CUDA C	PPCG	47.6	47.6
	QiMeng-Xpiler	100	98.2

- Compilation: 目标代码能编译； Computation: 输出通过单元测试
- SMT 主要修“能编译但算错”的局部错误， 显著提高 computation accuracy

QiMeng-Xpiler Performance

Correctness first; performance is reported only on valid cases



- 性能只统计 computation-correct 的 case
- 与 PyTorch / vendor backend 比较，说明转译后的 kernel 是否实用

QiMeng-Xpiler Case Study

FlashAttention shows the promise and limits of transcompilation

Table 10: Productivity Improvement by *QiMeng-Xpiler*.

Deformable Attention (~ 200LoCs)		CUDA C-> BANG C		C with VNNI -> CUDA C	
		Costs	Performance	Costs	Performance
Senior Coder	Manual	~6 d	100%	~1 d	100%
	Ours Time Saving	4.5 + 0.5 h ~28.8×	69.20%	2.1 h ~11.4×	132.50 %
Junior Coder	Manual	~30 d	49.85%	~3 d	75.76%
	Ours Time Saving	4.5 + 3 h ~96.0×	65.17%	2.1 h ~34.3×	132.50 %

Table 11: Normalized performance of *QiMeng-Xpiler* on FA1 and FA2 compared to vendor optimized implementations. **Configuration:** Batch Size = 8, Number of Heads = 16, Head Dimension = 32, Sequence Length = 1024.

Source	Operator	Target		
		HIP	BANG C	CUDA C
HIP	FA1	–	0.74	0.71
	FA2	–	0.76	0.74
BANG C	FA1	0.77	–	0.75
	FA2	0.78	–	0.61
CUDA C	FA1	0.69	0.70	–
	FA2	0.81	0.63	–

- FlashAttention: 性能只有baseline的60%-80%且是错误的结果
- 能生成可运行实现；复杂控制流和专用 layout 是主要瓶颈

```
for (int32_t i_m_5 = 0; i_m_5 < 8; ++i_m_5) {
  if ((((((int32_t *)xy_rounded)[i_m_5 + 8]) < 0) || (((int32_t *)height_width)[0] <= ((int32_t *)xy_rounded)[i_m_5 + 8]))) ||
    (((int32_t *)xy_rounded)[i_m_5 + 24] < 0) || (((int32_t *)height_width)[1] <= ((int32_t *)xy_rounded)[i_m_5 + 24]))) {
    for (int32_t i_d_7 = 0; i_d_7 < 512; ++i_d_7) {
      ((float *)corner_values)[(((i_m_5 * 512) + i_d_7) + 12288)] = 0.000000e+00f;
    }
  }
}
```

Figure 10: The complex control flow in Deformable Attention.

面向异构架构的自动化机器学习库生成

PerfDojo: Automated ML Library Generation for Heterogeneous Architectures

Andrei Ivanov
ETH Zurich
Zurich, Switzerland
andrei.ivanov@inf.ethz.ch

Marcin Chrapek
ETH Zurich
Zurich, Switzerland

Luca Benini
ETH Zurich
Zurich, Switzerland

Siyuan Shen
ETH Zurich
Zurich, Switzerland

Afif Boudaoud
ETH Zurich
Zurich, Switzerland

Torsten Hoefler
ETH Zurich
Zurich, Switzerland

Gioele Gottardo
ETH Zurich
Zurich, Switzerland

Timo Schneider
ETH Zurich
Zurich, Switzerland

PerfDojo: Background

Specialized High-Performance Accelerators

GPUs



nvidia.com

TPUs / ASICs

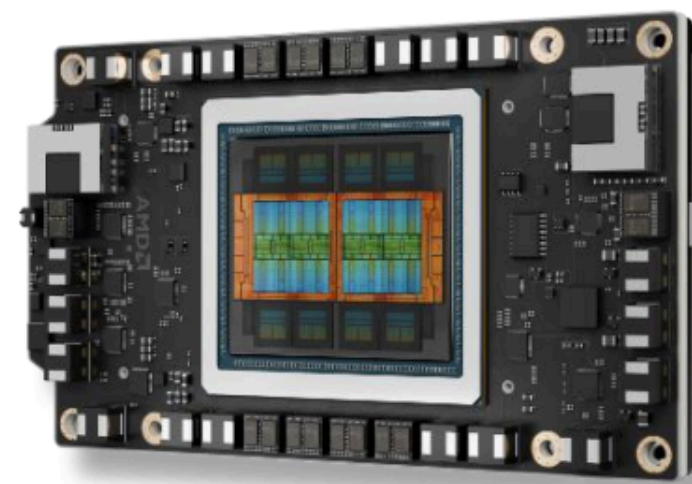


cloud.google.com

FPGAs



amd.com



amd.com

Evolving Processor Architectures

Advanced Vector Extensions (AVX)
Advanced Matrix Extensions (AMX)
ARM Neon
ARM Scalable Vector Extension (SVE)
ARM Scalable Matrix Extension (SME)
AMD Matrix Cores
NVIDIA Tensor Cores
Apple Matrix Coprocessor
RISC-V Stream Semantic Registers (SSR) [1]
RISC-V Floating-Point Repetition (FREP) [1]

- 硬件越来越异构：不同的硬件都需要不同优化
- 自动编译器通常到不了手写性能；手写优化又难以迁移和扩展

PerfDojo and PerfLLM

Separate the environment from the search policy

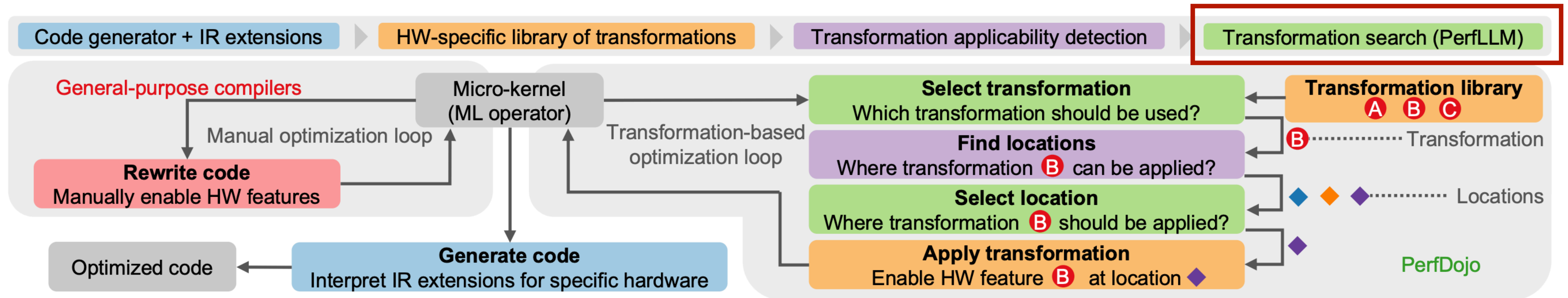
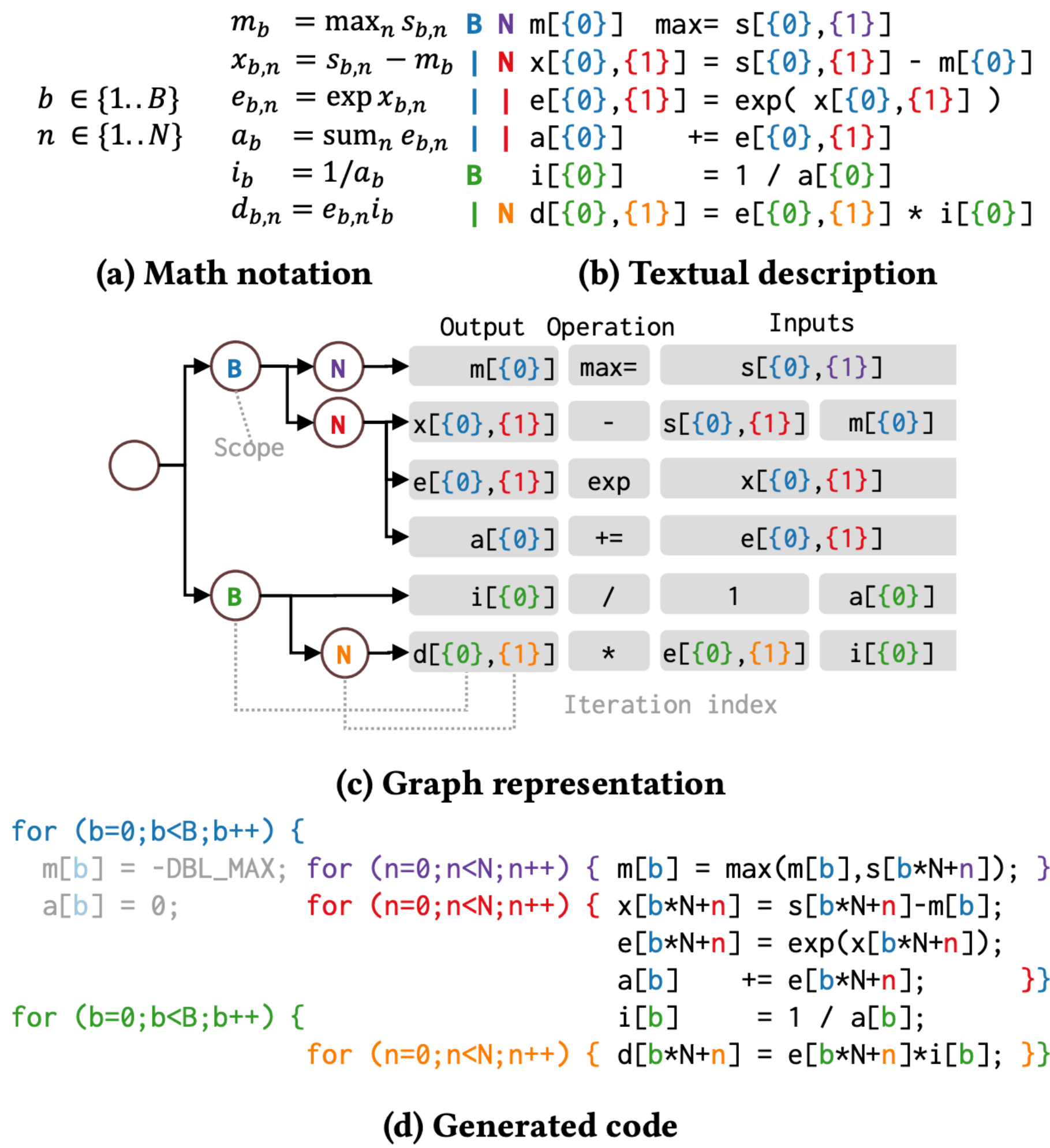


Figure 2: Manual vs. PerfDojo's transformation-centric optimization workflows.

- PerfDojo: 相当于specification, 为LLM提供安全的程序变换约定, 包含 IR、transformation 和 applicability
- PerfLLM: 训练一个模型, 用来在合法变换中做选择
- 性能闭环: 应用变换 → 编译运行 → 得到 reward → 更新搜索策略

PerfDojo Representation

One IR exposes scopes, operations and buffers to the optimizer



- IR 同时记录 loop scope、operation、buffer shape 和 storage
- transformation 直接修改 IR 结构，而不是让模型自由改代码文本
- 框架负责判断变换是否合法，并从 IR 生成目标代码

Figure 3: Softmax kernel representations.

Transformations and Preconditions

A move can be correct only after the right earlier move

第一步: `join_scopes: t,0`

它把两个操作的外层 `i0` 循环融合：

```
for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        t[i0*6+i1] = x[i0*6+i1];
for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        y[i0*6+i1] = t[i0*6+i1];
```

```
for (i0 = 0; i0 < 4; i0++) {
    for (i1 = 0; i1 < 6; i1++)
        t[i0*6+i1] = x[i0*6+i1];
    for (i1 = 0; i1 < 6; i1++)
        y[i0*6+i1] = t[i0*6+i1];
}
```

```
for (i0 = 0; i0 < 4; i0++) {  
    for (i1 = 0; i1 < 6; i1++)  
        t[i1] = x[i0*6+i1];  
    for (i1 = 0; i1 < 6; i1++)  
        y[i0*6+i1] = t[i1];  
}
```

✓ join_scopes: t, 0

```
reuse_dims: t,0
```

[illegible]

```
x f32 [4, 6] heap
t f32 [4, 6] heap
y f32 [4, 6] heap
```

Code:

4 6 $t[\{0\}, \{1\}] = x[\{0\}, \{1\}]$
4 6 $y[\{0\}, \{1\}] = t[\{0\}, \{1\}]$

```
reuse_dims: t, 0
```

Declarations: Incorrect

```
x f32 [4, 6] heap
t f32 [4:N, 6] heap
y f32 [4, 6] heap
```

Code:

4 6 $t[\{0\}, \{1\}] = x[\{0\}, \{1\}]$
4 6 $y[\{0\}, \{1\}] = t[\{0\}, \{1\}]$

Declarations: Correct

```
x f32 [4, 6] heap
t f32 [4, 6] heap
y f32 [4, 6] heap
```

Code:

```
4 6 t[{0},{1}] = x[{0},{1}]
| 6 y[{0},{1}] = t[{0},{1}]
```

// Incorrect!

```
for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        t[i1] = x[i0*6+i1];
for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        y[i0*6+i1] = t[i1];
```

Declarations: Correct

```
x f32 [4, 6] heap
t f32 [4:N, 6] heap
y f32 [4
```

Code:

```
4 6 t[{0},
| 6 y[{0},
```

```
for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        t[i1] = x[i0*6+i1];
```

```
for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        y[i0*6+i1] = t[i1];
```

Generated code

最左上角是：

</> C++

```
for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        t[i0*6+i1] = x[i0*6+i1];

for (i0 = 0; i0 < 4; i0++)
    for (i1 = 0; i1 < 6; i1++)
        y[i0*6+i1] = t[i0*6+i1];
```

</> C++

```
for (i0 = 0; i0 < 4; i0++) {  
    for (i1 = 0; i1 < 6; i1++)  
        t[i0*6+i1] = x[i0*6+i1];  
  
    for (i1 = 0; i1 < 6; i1++)  
        y[i0*6+i1] = t[i0*6+i1];  
}
```

第二步: reuse_dims: t,0

融合之后，每次 `i0` 只需要保存当前这一行：

</> C++

```
for (i0 = 0; i0 < 4; i0++) {  
    for (i1 = 0; i1 < 6; i1++)  
        t[i1] = x[i0*6+i1];  
  
    for (i1 = 0; i1 < 6; i1++)  
        y[i0*6+i1] = t[i1];  
}
```

↑ transformation 的合法性依赖

当前 IR 状态。

- **例子：**先 loop fusion，再 buffer reuse；否则语义错误。

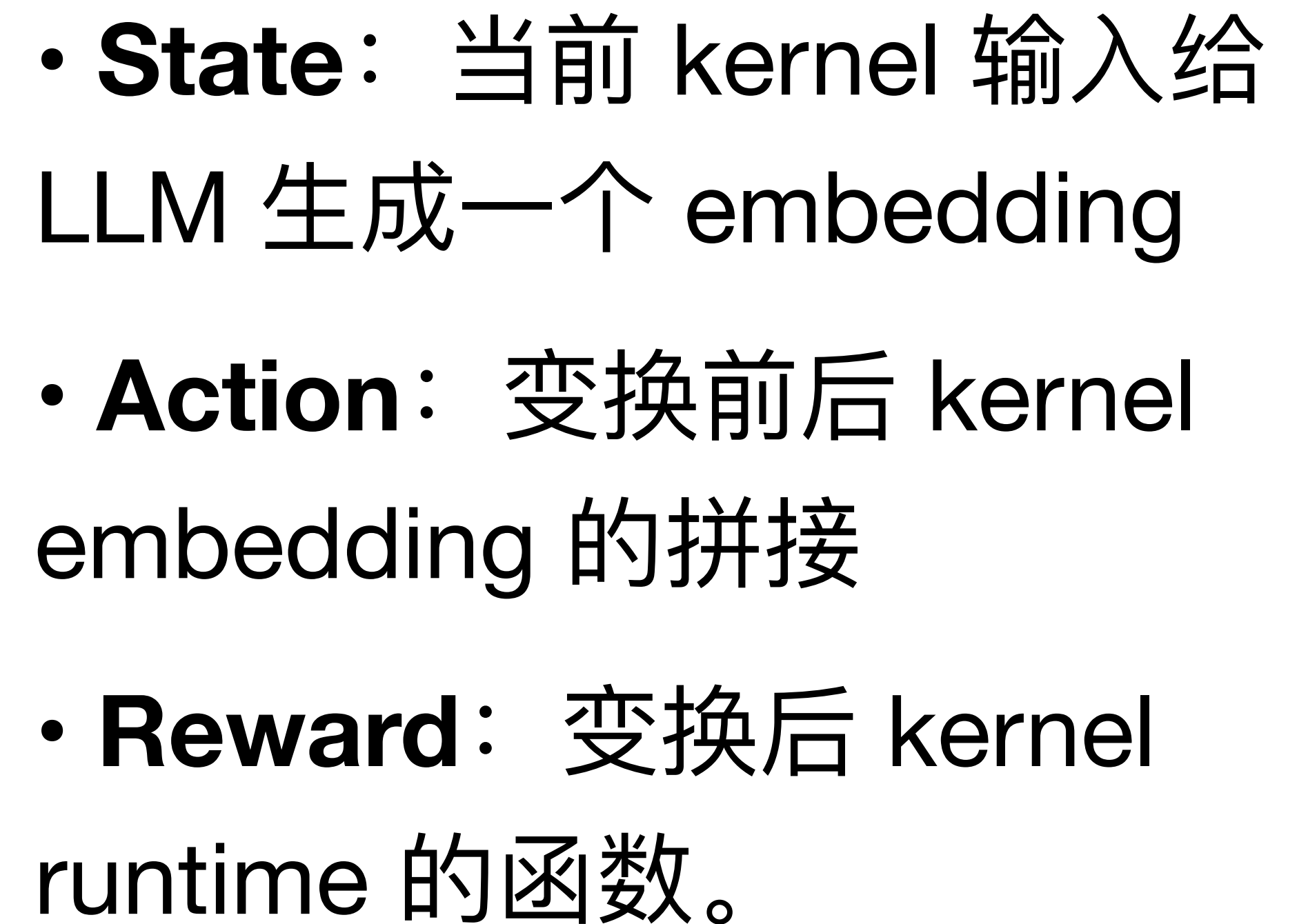
Transformation Library

27 transformations with applicability detection

Transformation	Description	Example before	Example after	Transformation	Description	Example before	Example after
swap_ops	Swaps adjacent operations or scopes	<code>N y[{0}]=x[{0}] z=q</code>	<code>z=q N y[{0}]=x[{0}]</code>	reuse_dims	Avoids materializing a dimension	<code>x f32 [M, K] heap</code>	<code>x f32 [M:N, K] heap</code>
tile_scope	Tiles a scope	<code>10 y+=x[{0}]</code>	<code>5 2 y+=x[{0}]*2+{1}]</code>	reuse_buffers	Shares storage between arrays	<code>x f32[M,K] heap y f32[M,K] heap</code>	<code>x f32[M,K] heap->x,y</code>
tile_buffer	Tiles a buffer dimension	<code>M N y+=x[{0}*N+{1}]</code>	<code>M N y+=x[{0},{1}]</code>	swap_dims	Changes the memory layout of an array	<code>x f32 [M, N, K] heap</code>	<code>x f32 [M, K, N] heap</code>
swap_scopes	Reorders adjacent scopes	<code>M N y+=x[{0},{1}]</code>	<code>N M y+=x[{1},{0}]</code>	increment_padding	Increases padding	<code>x f32 [M,K+2] heap</code>	<code>x f32 [M,K+3] heap</code>
join_scopes	Joins the bodies of adjacent scopes	<code>M y[{0}]=x[{0}] M p[{0}]=q[{0}]</code>	<code>M y[{0}]=x[{0}] p[{0}]=q[{0}]</code>	repeat_padding	Doubles the padding	<code>x f32 [M+3,K+3] heap</code>	<code>x f32 [M+6,K+3] heap</code>
create_temporary	Introduces a temporary array copy	<code>M y+=x[{0}]</code>	<code>M t[{0}]=x[{0}] M y+=t[{0}]</code>	move_to_stack	Moves an array to the stack	<code>x f32 [N] heap</code>	<code>x f32 [N] stack</code>
27 transformations with applicability detection							
unroll_scope	Unrolls a scope	<code>N y[{0}]=x[{0}]</code>	<code>N:u y[{0}]=x[{0}]</code>	vectorize_buffer	Vectorizes storage of a buffer	<code>x f32 [4,16] stack</code>	<code>x f32x16 [4,16] stack</code>
enable_frep	Instantiates a scope using FREP	<code>N y+= \ x[{0} s:5,d:1,h:2]</code>	<code>N:f y+= \ x[{0} s:5,d:1,h:2]</code>	enable_ssr	Enables SSR for an array access	<code>M N y+= x[{0},{1}]</code>	<code>M N y+= x[{0},{1} s:5,d:1]</code>
parallelize	Parallelizes a scope using OpenMP	<code>N y[{0}]=x[{0}]</code>	<code>N:p y[{0}]=x[{0}]</code>	increase_ssr_depth	Increases the SSR depth	<code>M N y+= \ x[{0},{1} s:5,d:1]</code>	<code>M N y+= \ x[{0},{1} s:5,d:2]</code>
parallelize_grid	Parallelizes a scope as a CUDA grid	<code>N y[{0}]=x[{0}]</code>	<code>N:g y[{0}]=x[{0}]</code>	merge_ssr	Merges multiple SSR accesses into one	<code>M y+=x[{0} s:5,d:1] M y+=z[{0} s:7,d:1]</code>	<code>M y+=x[{0} s:5,d:1] M y+=z[{0} s:5,d:1]</code>
parallelize_block	Parallelizes a scope as a CUDA block	<code>N y[{0}]=x[{0}]</code>	<code>N:b y[{0}]=x[{0}]</code>	activate_ssr	Activates an SSR configuration	<code>N y+= \ x[{0} s:5,d:1]</code>	<code>N y+= \ x[{0} s:5,d:1,h:2]</code>
parallelize_warp	Parallelizes a scope as a CUDA warp	<code>N y[{0}]=x[{0}]</code>	<code>N:w y[{0}]=x[{0}]</code>				
vectorize_scope	Vectorizes a scope	<code>4 y[{0}]=x[{0}]</code>	<code>4:v y[{0}]=x[{0}]</code>				

- **Transformation library:** 包含 tiling、fusion、reuse、vectorization、parallelization 等。
- **Applicability detection:** 系统判断某个变换能否用于当前位置。

Kernels become states; transformations become candidate actions



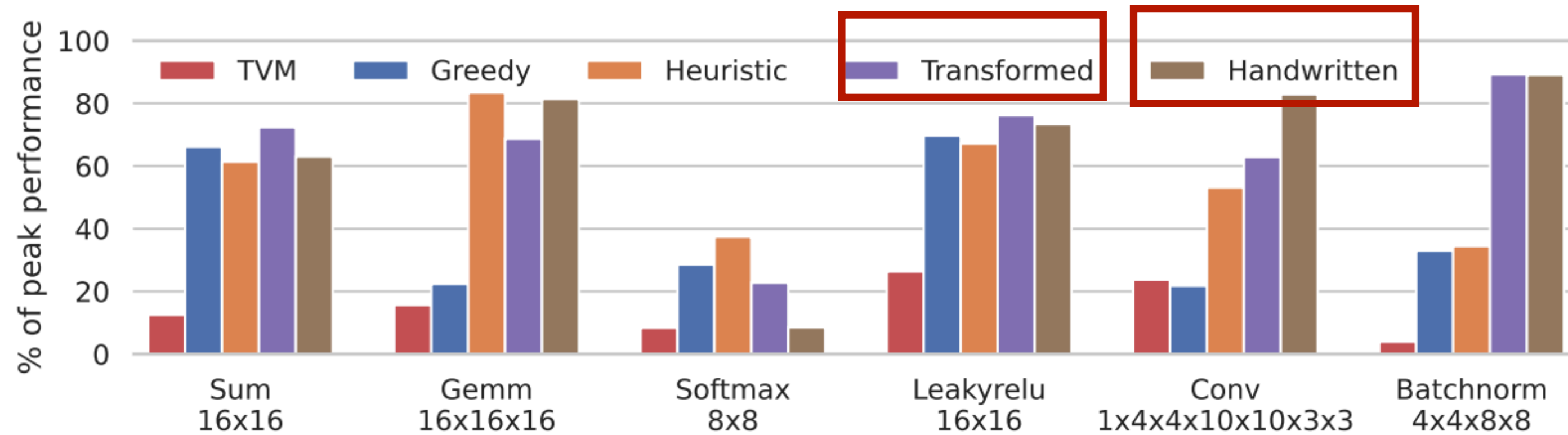
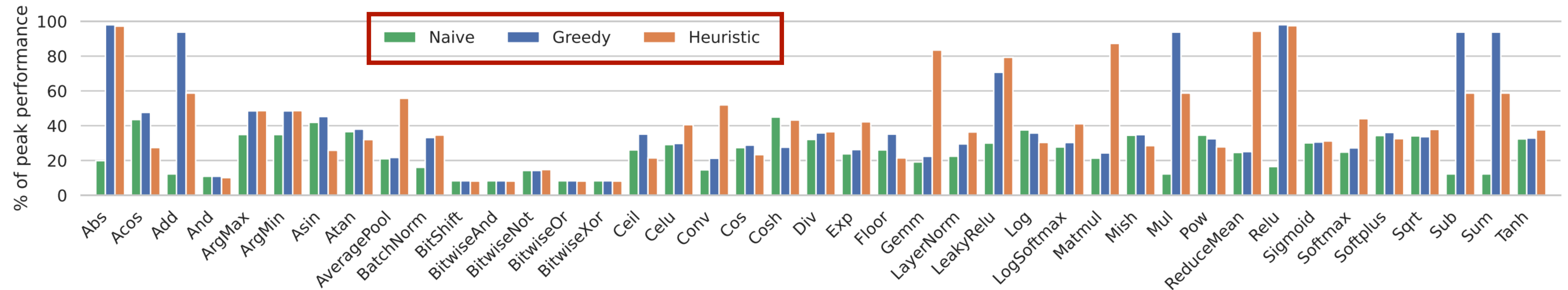
Evaluation Structure

Three experimental objects, not one unified benchmark

- 1. 验证 PerfDojo提供的transformation library 能表达新硬件优化
- 2. 用 heuristic search 评估传统搜索下的优化能力
- 3. 验证PerfLLM: 去掉人工性能 heuristic, 测试能否靠 RL 学策略
- 但三部分不是同样的硬件设定, 更像分模块能力展示

Evaluation: Snitch

PerfDojo: Heuristics encode hardware expertise



- 验证 PerfDojo 能表达硬件优化。
- **Heuristic**: 专家规则编码硬件知识, 不是 learned policy。

Evaluation: x86 + Heuristics

Search evaluates many transformation sequences under a fixed budget

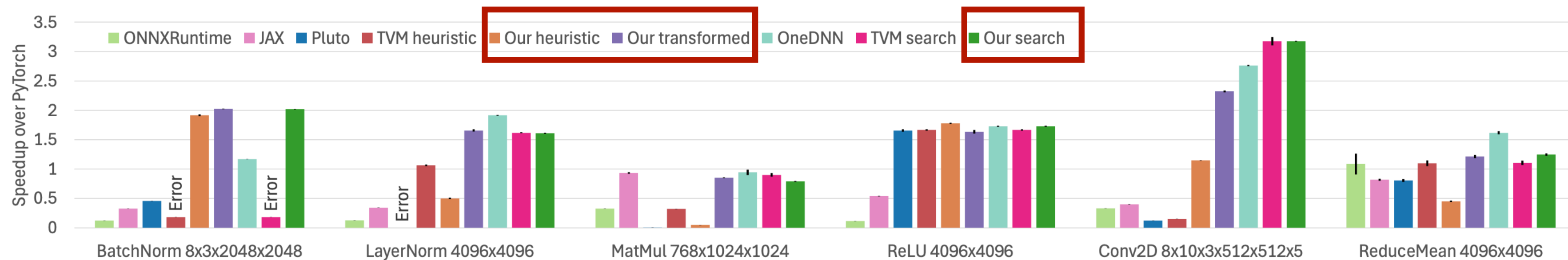


Figure 10: Kernel performance across various frameworks and libraries on x86. The *heuristic* version performs a single program evaluation pass, whereas the *search* version continues until reaching a limit of 1000 evaluations, after which the search terminates. The *transformed* version is derived by applying transformations manually.

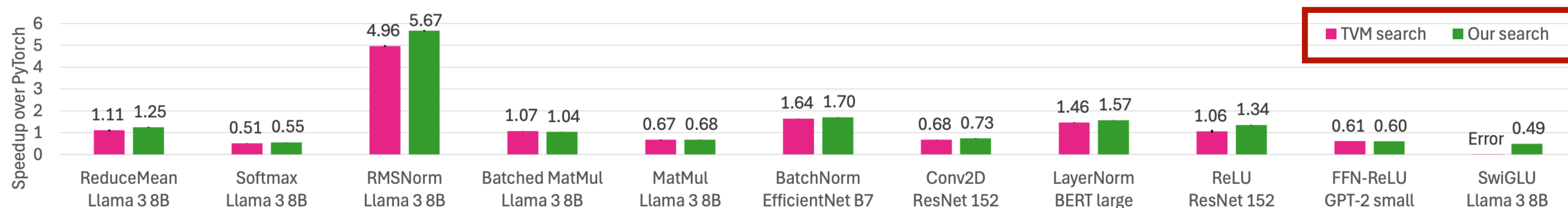
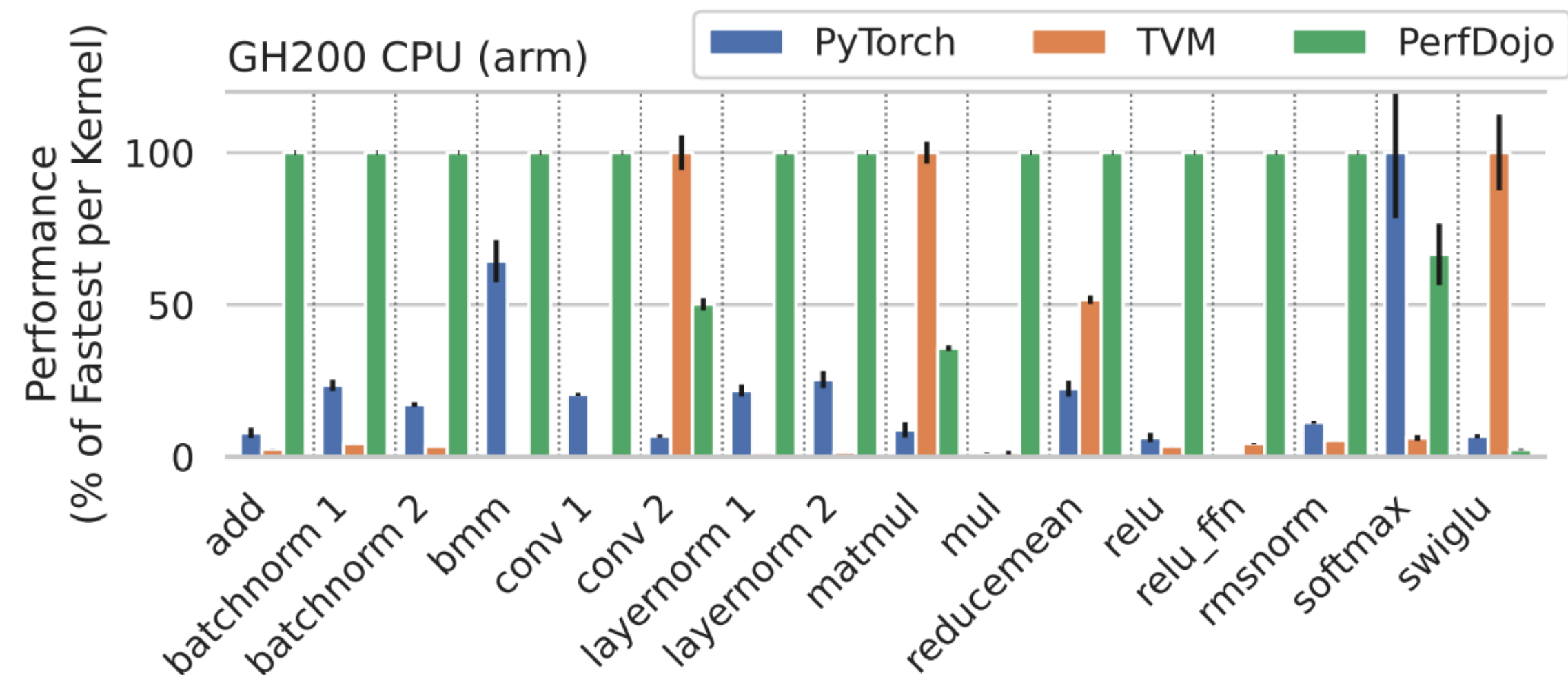
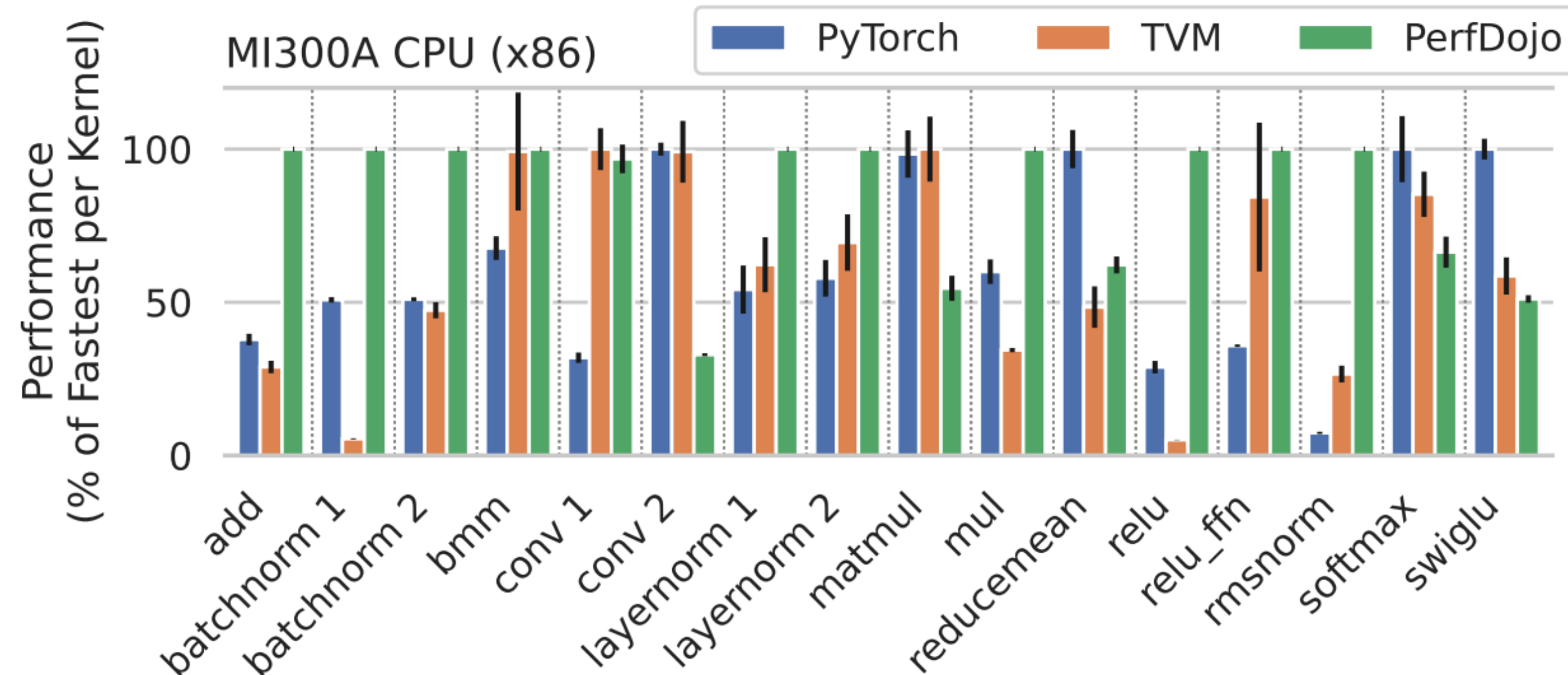


Figure 11: Kernel performance across various shapes in existing models after 1000 auto-tuning evaluations on x86. Excluding SwiGLU, where the TVM auto-scheduler fails to produce a valid schedule, our approach achieves a 7.6% geometric mean speedup over it.

- 搜索 transformation sequences, runtime 决定扩展方向。
- 结论：在 x86 上 heuristic search 与现有系统有竞争力。

Evaluation: PerfLLM / No Heuristics

Learned policy can discover competitive kernels, but tuning is expensive



- 在 MI300A/GH200 上，PerfLLM 用 learned policy 搜索 kernel 实现
- 不预先写硬件性能 heuristic，只依赖 transformation library 和 runtime reward
- 能找到有竞争力的实现

Summary

- 共同点：将Kernel生成拆成受约束步骤，候选生成 / 合法性检查 / 局部修复 / 性能反馈
- LLM 负责提出候选；编译器、SMT、测试负责把候选拉回正确空间
- 从 prompt engineering 走向可验证、可迭代的 harness framework